

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

CE-403: Database Administration (PostgreSQL)

Elchin Salahzada

Lecture 5

CE-403: Database Management Systems
Baku Higher Oil School : Computer Engineering

Database Design & ER Modeling

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Introduction to Database Design Process
- Entities, Attributes, and Relationships
- Entity-Relationship (ER) Diagrams
- Cardinality & Participation Constraints
- Normalization – 1NF, 2NF, 3NF
- BCNF & Denormalization
- Lab Activity

Learning objectives

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Explain the database design process from requirements to implementation
- Identify entities, attributes, and relationships in a real-world scenario
- Construct ER diagrams to represent a database's structure visually
- Apply cardinality and participation constraints to define relationships accurately
- Normalize a database schema through 1NF, 2NF, and 3NF to eliminate redundancy
- Evaluate when to apply BCNF or denormalize a schema for performance

Database Design Process

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Database design transforms real-world requirements into a structured database schema
- Design Process Stages:
  - Requirements Analysis – Understand what data needs to be stored and how it will be used
  - Conceptual Design – Create an ER model (entities, attributes, relationships)
  - Logical Design – Convert ER model into relational schema (tables, keys)
  - Normalization – Refine schema to reduce redundancy
  - Physical Design – Define storage, indexing, and performance considerations
- Good design ensures data integrity, minimal redundancy, and efficient querying
- Poor design leads to anomalies: insertion, update, and deletion anomalies

Entities, Attributes, Relationships

- Entity – A real-world object or concept represented in the database (e.g., Student, Course, Employee)
- Entity Set – A collection of similar entities (e.g., all Students)
- Attribute – A property or characteristic of an entity (e.g., Student → name, age, email)
- Types of Attributes:
  - Simple – cannot be divided further (e.g., age)
  - Composite – can be broken into parts (e.g., address → street, city, zip)
  - Derived – calculated from other attributes (e.g., age from date of birth)
  - Multivalued – can hold multiple values (e.g., phone numbers)
- Relationship – An association between two or more entities (e.g., Student enrolls in Course)
- Relationships can also have their own attributes (e.g., enrollment date)

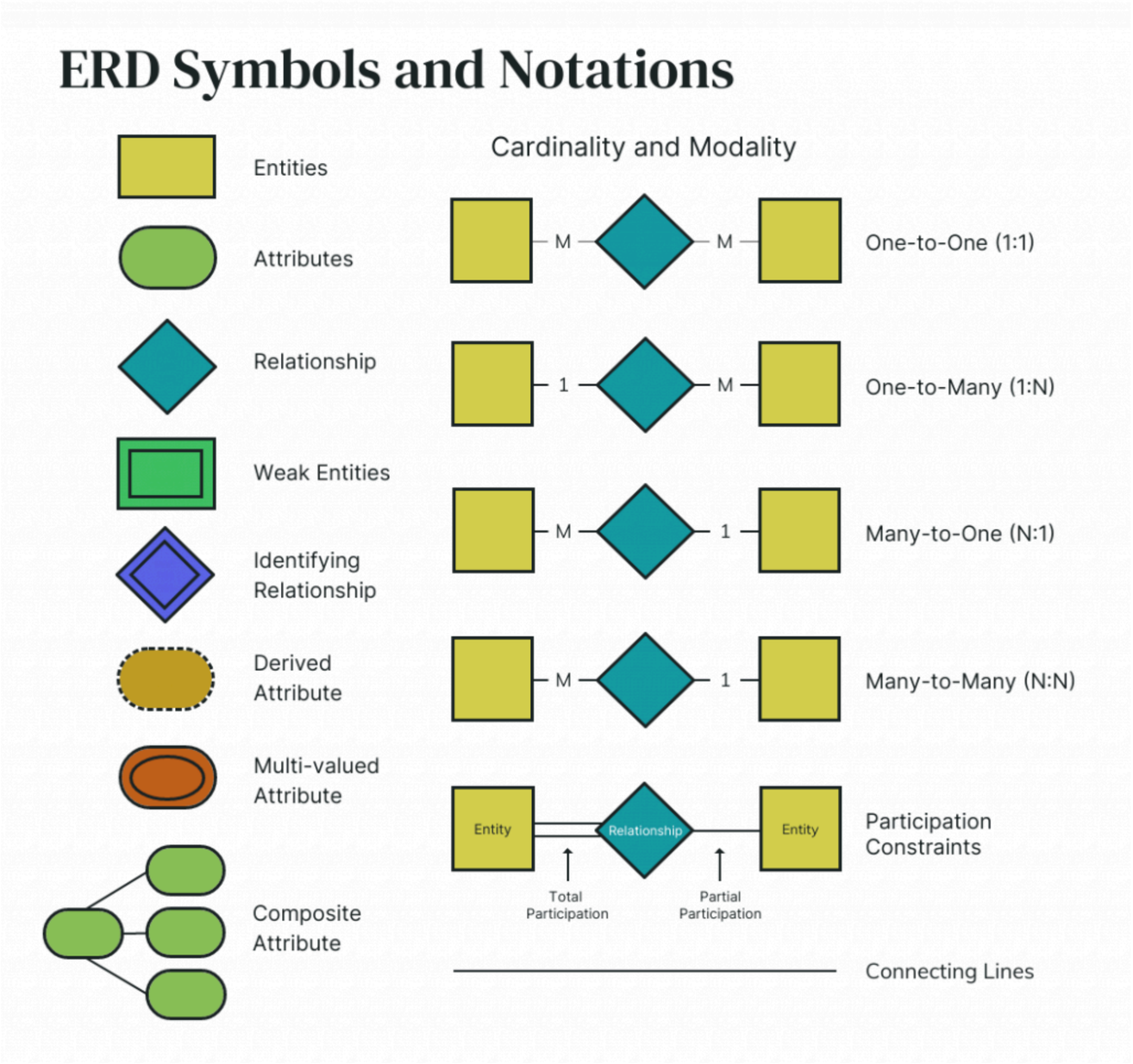
Entity-Relationship Diagrams

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- An ER diagram is a visual representation of entities, attributes, and relationships in a database
- Standard Notation:
  - Rectangle – Entity
  - Oval – Attribute
  - Diamond – Relationship
  - Lines – Connect entities to attributes and relationships
- Example:
  - Student — enrolls in — CourseStudent (rectangle) connects via Enrolls (diamond) to Course (rectangle)
  - Attributes like name, student_id attach to Student as ovals
- Underlined attribute – indicates the primary key
- ER diagrams help visualize the schema before implementation, making it easier to communicate design with stakeholders
- Tools: dbdiagram.io, Lucidchart, draw.io, pgAdmin's ER tool

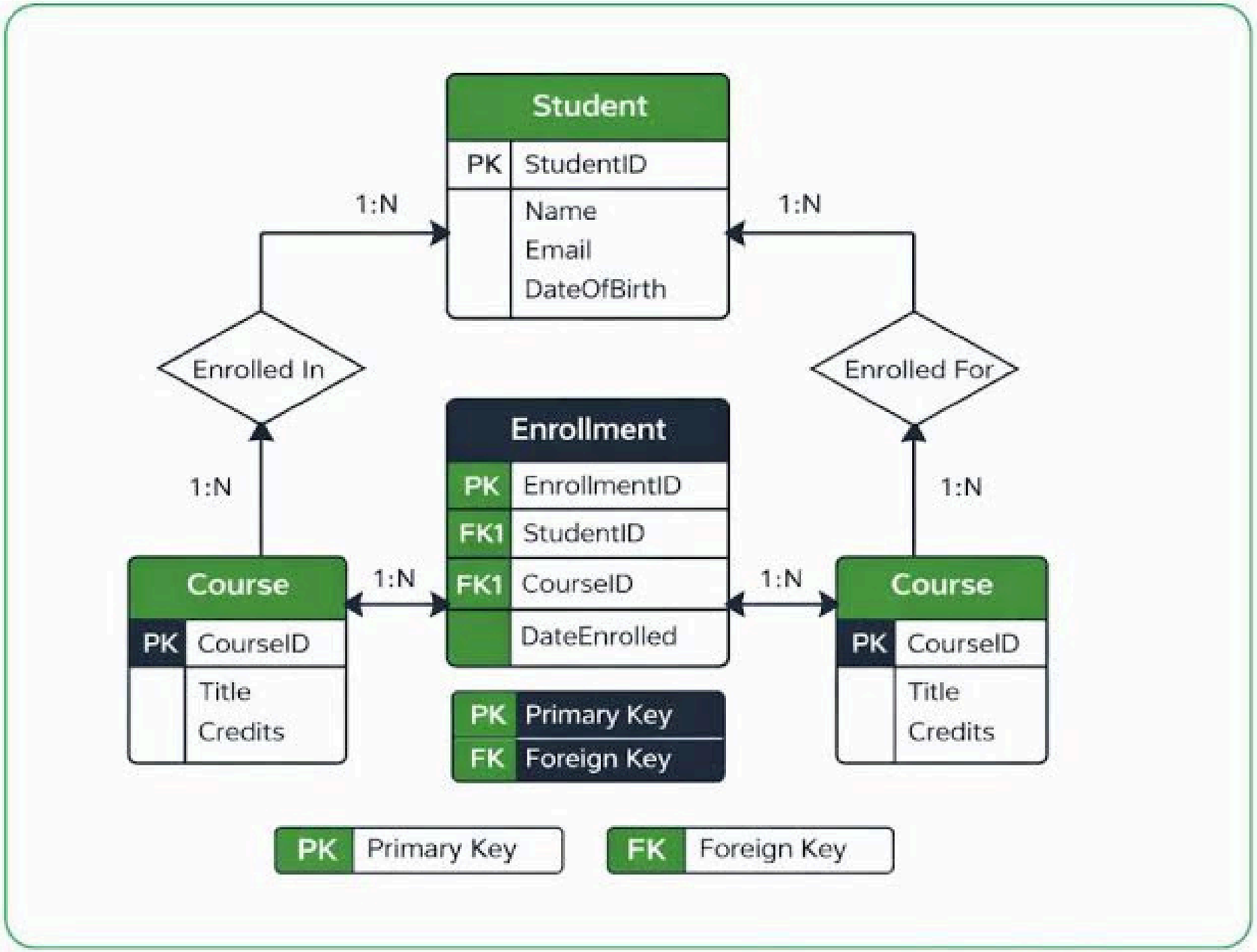
Entity-Relationship Diagrams

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL



Entity-Relationship Diagrams

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL



Cardinality & Participation Constraints

- Cardinality & Participation Constraints
- Cardinality – defines the number of instances of one entity that can relate to instances of another
- Types of Cardinality:
  - One-to-One (1:1) – e.g., one Person has one Passport
  - One-to-Many (1:N) – e.g., one Department has many Employees
  - Many-to-Many (M:N) – e.g., Students enroll in many Courses, Courses have many Students
- Participation Constraint – defines whether entity participation in a relationship is required
- Total Participation – every entity instance must participate (double line in ER diagram)
- Partial Participation – entity instance may or may not participate (single line)
- Example: Every Employee must belong to a Department (total participation), but a Department may have zero employees (partial participation)
- Many-to-Many relationships are implemented using a junction/bridge table

Normalization – 1NF, 2NF, 3NF

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Normalization organizes data to reduce redundancy and prevent anomalies
- First Normal Form (1NF)
  - Each column contains indivisible values
  - No repeating groups or multi-valued columns
- Second Normal Form (2NF)
  - Must be in 1NF
  - No partial dependency — non-key attributes must depend on the whole primary key (relevant for composite keys)
- Third Normal Form (3NF)
  - Must be in 2NF
  - No transitive dependency — non-key attributes must depend only on the primary key, not on other non-key attributes
- Example progression:
  - 1NF: Split multi-valued phone numbers into separate rows
  - 2NF: Separate order details that depend only on product_id, not the full composite key
  - 3NF: Move dept_name (dependent on dept_id) into its own departments table

BCNF & Denormalization

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Boyce-Codd Normal Form (BCNF)
  - A stricter version of 3NF
  - For every functional dependency X → Y, X must be a super key
  - Resolves rare edge cases where 3NF still allows anomalies (overlapping candidate keys)
- Example: If a table has a dependency where a non-super-key attribute determines another attribute, it violates BCNF and must be split into separate tables
- Denormalization
  - The process of intentionally introducing redundancy to improve read performance
  - Combines normalized tables back for faster queries (fewer joins)
- When to Denormalize:
  - Read-heavy applications (reporting, analytics/OLAP systems)
  - Performance is more critical than storage efficiency
- Trade-off: Normalization → data integrity; Denormalization → query performance

CASE when

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Creating several columns for visualizations based on given conditions, does not change table itself.
  - select s.id, s.name,
  - case when s.salary>2500 then '>2500'
  - when s.salary>1500 then '>2500'
  - when s.age>20 then '>2500'
  - when s.name like '%N%' then '>2500'
  - else '<2500' end as check_salary,

  - case when s.salary>2500 then '>2500'
  - end as second_condition,
  - from l6.students s;

Lecture Summary

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Database design follows a process: requirements → conceptual → logical → normalization → physical design
- Entities, attributes, and relationships form the building blocks of a data model
- ER diagrams visually represent entities, attributes, and relationships using standard notation
- Cardinality (1:1, 1:N, M:N) and participation constraints (total/partial) define relationship rules
- Normalization (1NF → 2NF → 3NF) removes redundancy by eliminating partial and transitive dependencies
- BCNF handles edge cases 3NF misses; denormalization trades some redundancy for query performance
- Good design balances data integrity with practical performance needs

Hands-on

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Scenario: Design a database for a university system (Students, Courses, Enrollments, Instructors)
- Identify entities, attributes, and relationships from the scenario (write them out)
- Draw an ER diagram (using dbdiagram.io, draw.io, or pen & paper) showing entities, attributes, primary keys, and relationships
- Define cardinality for each relationship (e.g., Student M:N Course, Instructor 1:N Course)
- Mark total vs. partial participation for each entity in its relationships
- Convert the ER diagram into a set of unnormalized tables (write sample data with repeating/multi-valued columns)
- Apply 1NF – eliminate repeating groups/multi-valued attributes
- Apply 2NF – remove partial dependencies (create separate tables where needed)
- Apply 3NF – remove transitive dependencies (e.g., separate instructor_name into its own table)
- Implement the final normalized schema using CREATE TABLE statements in PostgreSQL

Next Lecture

- Indexes & Query Optimization